

The login page needs a `POST /api/auth/login` endpoint that returns a JWT token, and every other endpoint should require that token in the `Authorization` header

Dashboard

The dashboard needs endpoints to fetch summary metrics, the active field's weed density grid for the map, and the latest run's species breakdown. It also needs real-time device status (connectivity and working/idle state), which should either be polled on an interval or served via WebSocket. The field management actions — listing fields, creating a field with dimensions and partition config, and deleting a field — are also initiated from the dashboard's field selector.

Analytics

The analytics page needs a paginated session log endpoint that accepts filter parameters for device ID, field, and month. Clicking a run opens the detail page, which requires a single-run endpoint returning the field grid distribution, species breakdown, device summary (total weeds, photos, run time), and the full detection log (grid position, coordinates, original image URL, annotated image URL) — also paginated.

Device Control

This page has the most endpoints. For device selection it needs the device list with connectivity and status. Weed detection requires a start endpoint (accepting grid position or route name) and a stop endpoint, plus a WebSocket or polling endpoint to stream real-time device position and scanned cell data back to the field map. Manual control needs individual move/turn command endpoints. Route management needs list routes, create route (with an ordered instruction list), and delete route endpoints.

Settings

Settings needs a fetch endpoint to pull current device configuration values, and a send/update endpoint to push the edited config back to the device. Both are scoped to a specific device ID.

Device Control — Weed Detection (updated)

Method	Endpoint	Purpose
POST	<code>/api/devices/:id/detection/start</code>	Start weed detection — body: { mode: "grid" "route", grid_x, grid_y, distance } or { mode: "route", route_id }
POST	<code>/api/devices/:id/detection/stop</code>	Stop weed detection mid-run
GET	<code>/api/devices/:id/detection/status</code>	Poll current detection state — { status: "running" "idle" "completed" "stopped", cells_total, cells_scanned, weeds_found, device_position: { x, y }, last_updated }
GET	<code>/api/devices/:id/detection/grid</code>	Poll current density grid state — returns scanned cell results so far, used to update the field map in real time

How polling replaces the WebSocket

The frontend calls `GET /api/devices/:id/detection/status` and `GET /api/devices/:id/detection/grid` on a fixed interval (e.g. every 2 seconds) while detection is running. When `status` returns `"completed"` or `"stopped"`, the frontend stops polling and returns the weed detection panel to its idle state, updating the last run note with the finish timestamp from the response.

The full updated endpoint table with no WebSocket anywhere:

Authentication

Method	Endpoint	Purpose
POST	<code>/api/auth/login</code>	Validate username & password, return JWT

POST	<code>/api/auth/logout</code>	Invalidate token
GET	<code>/api/auth/me</code>	Get current logged-in user info

Fields

Method	Endpoint	Purpose
GET	<code>/api/fields</code>	List all fields
POST	<code>/api/fields</code>	Create a field
GET	<code>/api/fields/:id</code>	Get a single field
PUT	<code>/api/fields/:id</code>	Update a field
DELETE	<code>/api/fields/:id</code>	Delete a field

Devices

Method	Endpoint	Purpose
GET	<code>/api/devices</code>	List all devices with connectivity and status
GET	<code>/api/devices/:id</code>	Get a single device's details and status
GET	<code>/api/devices/:id/ping</code>	Check if device is online

Dashboard

Method	Endpoint	Purpose
--------	----------	---------

GET	<code>/api/dashboard/metrics?field_id= =</code>	Total weeds detected, total runs, active device count
GET	<code>/api/dashboard/species-breakdown?field_id=&run_id=latest</code>	Species breakdown for pie chart
GET	<code>/api/dashboard/density-map?field_id=&run_id=latest</code>	Grid weed density for latest run
GET	<code>/api/dashboard/runs-chart?field_id=&period=30d</code>	Weeds per run over time for line chart

Analytics

Method	Endpoint	Purpose
GET	<code>/api/runs?device_id=&field_id=&month=&page=&limit=</code>	Paginated filtered session log
GET	<code>/api/runs/:run_id</code>	Single run header
GET	<code>/api/runs/:run_id/density-map</code>	Weed density grid for a specific run
GET	<code>/api/runs/:run_id/species</code>	Species breakdown for a specific run
GET	<code>/api/runs/:run_id/device-summary</code>	Device stats for a specific run
GET	<code>/api/runs/:run_id/detection-logs?page=&limit=</code>	Paginated detection log with photo URLs

Media

Method	Endpoint	Purpose
GET	<code>/api/media/original/:filename</code>	Serve original detection photo

GET	<code>/api/media/annotated/:filename</code>	Serve annotated detection photo
POST	<code>/api/media/upload</code>	Upload image, annotated image, prediction results, deviceId, timestamp, grid position

Device Control — Weed Detection

Method	Endpoint	Purpose
POST	<code>/api/devices/:id/detection/start</code>	Start weed detection
POST	<code>/api/devices/:id/detection/stop</code>	Stop weed detection
GET	<code>/api/devices/:id/detection/status</code>	Poll detection state, device position, cells scanned, weeds found
GET	<code>/api/devices/:id/detection/grid</code>	Poll live grid cell results to update field map

Device Control — Manual Control

Method	Endpoint	Purpose
POST	<code>/api/devices/:id/move</code>	Send move command — body: { <code>command</code> : "forward" "backward" "left" "right", <code>value</code> : <metres or degrees> }

Device Control — Routes

Method	Endpoint	Purpose
GET	<code>/api/routes?device_id=</code>	List all saved routes
GET	<code>/api/routes/:id</code>	Get a single route with instruction list

POST	/api/routes	Save a new route
PUT	/api/routes/:id	Update a route
DELETE	/api/routes/:id	Delete a route

Settings

Method	Endpoint	Purpose
GET	/api/devices/:id/settings	Fetch current device config
PUT	/api/devices/:id/settings	Push updated config to device

Summary by page

Page	Endpoints used
Login	/api/auth/login
Dashboard	/api/fields , /api/devices , /api/dashboard/metrics , /api/dashboard/species-breakdown , /api/dashboard/density-map , /api/dashboard/runs-chart
Analytics session log	/api/runs
Analytics run detail	/api/runs/:id , /api/runs/:id/density-map , /api/runs/:id/species , /api/runs/:id/device-summary , /api/runs/:id/detection-logs , /api/media/original/:f , /api/media/annotated/:f
Device control	/api/devices , /api/devices/:id/detection/start , /api/devices/:id/detection/stop , /api/devices/:id/detection/status , /api/devices/:id/detection/grid , /api/devices/:id/move , /api/routes (CRUD)

Settings `/api/devices/:id/settings` (GET + PUT)

Polling implementation note

When the user clicks Start weed detection, the frontend begins calling `/api/devices/:id/detection/status` every 2 seconds. Each response includes a `status` field — while it returns `"running"`, the frontend keeps polling and uses the `device_position` and `cells_scanned` values to update the map and counters. When `status` returns `"completed"` or `"stopped"`, polling stops, the panel resets to idle, and the `finished_at` or `stopped_at` timestamp from the response populates the last run note. The `/api/devices/:id/detection/grid` endpoint can be polled at a slightly longer interval (e.g. every 3–4 seconds) since the full grid update is heavier than the status check.

Device Manager page

Method	Endpoint	Purpose
GET	<code>/api/devices</code>	Fetch all devices for the table — returns device name, connectivity, status, created date
GET	<code>/api/devices?name=&created_date=</code>	Filter devices by name (partial match) and/or created date
GET	<code>/api/devices/check-name?name=</code>	Check if a device name already exists — returns <code>{ available: true false }</code> — called on every keystroke with debounce during creation
POST	<code>/api/devices</code>	Create a new device — body: <code>{ name, device_id, server_url, serial_port, serial_baud_rate, camera_index, confidence_threshold }</code>

DELETE `/api/devices/:id`

Delete a device by ID — called after user confirms in the delete modal

Request and response details

GET `/api/devices` response:

```
[
  {
    "id": "abc123",
    "name": "DEV-001",
    "connectivity": "online",
    "status": "idle",
    "created_date": "2025-01-10"
  }
]
```

GET `/api/devices/check-name?name=DEV-004` response:

```
{ "available": true }
```

POST `/api/devices` request body:

```
{
  "name": "DEV-004",
  "device_id": "DEV-004",
}
```

```
"server_url": "http://192.168.1.13:5000",  
"serial_port": "/dev/ttyUSB3",  
"serial_baud_rate": 115200,  
"camera_index": 0,  
"confidence_threshold": 0.75  
}
```

DELETE `/api/devices/:id` response:

```
{ "deleted": true }
```

Notes

The filter on the page works with two optional query parameters on the same `GET /api/devices` endpoint — `name` for partial text search and `created_date` for exact date match. Both can be used independently or together, which matches the "individual or combination" filter behaviour in the wireframe.

The `GET /api/devices/check-name` endpoint should be lightweight and fast since it is called on every keystroke with a short debounce. It only needs to check name uniqueness — no other data is returned.

All endpoints require the JWT token in the `Authorization: Bearer <token>` header.

The `connectivity` and `status` fields on `GET /api/devices` should reflect live state — the backend should derive connectivity from the most recent device ping timestamp, and status from whether a detection run is currently active on that device.